Technical Specification: ESP32 Direct Communication Service

Dart Client Implementation

March 18, 2026

Abstract

This document outlines the architecture, connection flow, and communication protocol for the `EspDirectService`, a Dart-based WebSocket client designed to communicate directly with an ESP32 microcontroller. The service handles authentication, real-time valve control, telemetry data consumption, and remote WiFi configuration.

Contents

1 System Overview **2**

2 Connection and State Management **2**
2.1 Connection Parameters 2
2.2 Reactive Streams 2

3 Protocol and Message Flow **2**
3.1 Phase 1: Connection and Authentication 2
3.2 Phase 2: Data Request 3

4 Hardware Control API **3**
4.1 Valve Manipulation 3
4.2 WiFi Provisioning 3

5 Cleanup and Resource Management **3**

1 System Overview

The `EspDirectService` acts as the primary bridge between the Flutter/Dart application and the ESP32 hardware. It is designed to interface with the ESP32's internal WebSocket server running on an HTTP daemon (port 80) at the URI path `/ws`.

The service is implemented as a Singleton to prevent multiple concurrent WebSocket connections and to provide a centralized state management hub for device telemetry.

2 Connection and State Management

2.1 Connection Parameters

By default, the service connects assuming the ESP32 is operating in Access Point (AP) mode with the following defaults:

- **Default IP Address:** `192.168.4.1`
- **Default Port:** `80`
- **WebSocket Path:** `/ws`

2.2 Reactive Streams

The service utilizes asynchronous Dart `StreamController` objects to broadcast incoming data across the application. Clients can subscribe to the following streams:

1. `connectionStream`: Broadcasts boolean state changes regarding socket connectivity.
2. `deviceInfoStream`: Emits authentication responses containing the `device_id`.
3. `valveDataStream`: Delivers full operational state data (angles, limits, controller status).
4. `errorStream`: Emits critical hardware or validation errors broadcasted by the ESP32.

3 Protocol and Message Flow

Communication is entirely bidirectional and JSON-based. The exact message flow relies heavily on an authentication gate.

3.1 Phase 1: Connection and Authentication

Upon successfully opening the WebSocket connection, the client is marked as *connected* but *unauthenticated*. The ESP32 firmware (`websocket_state_fn.c`) will ignore all operational commands until a valid passkey is provided.

The client sends an authentication request:

```
{
  "event": "request_device_info",
  "passkey": "YOUR_PASSKEY"
}
```

Listing 1: Authentication Request

If the passkey matches the ESP32's internal `CONFIG_WS_PASSKEY_VALUE`, the ESP32 responds with device information, and the client marks itself as authenticated:

```
{
  "event": "device_info",
  "timestamp": "2023-10-27 10:00:00",
  "device_id": "DEVICE_ID"
}
```

Listing 2: Authentication Response

3.2 Phase 2: Data Request

Once authenticated, the client requests the current state of the valve hardware. This requires passing the authenticated `device_id` to ensure proper routing.

```
{
  "event": "device_basic_info",
  "data": {
    "user_id": "user001",
    "device_id": "dev0016"
  }
}
```

Listing 3: Device Basic Info Request

The ESP32 responds with a `valve_data` event containing nested JSON objects detailing the controller state, current physical valve data, and limit configurations.

4 Hardware Control API

4.1 Valve Manipulation

The client can manually dictate the physical angle of the connected valve. This command disables any active scheduling or sensor-based control on the ESP32 hardware, forcing manual override mode.

```
{
  "event": "set_valve_basic",
  "valve_data": {
    "set_angle": true,
    "angle": 45
  }
}
```

Listing 4: Set Valve Angle Command

Note: The Dart implementation safeguards this input by utilizing `angle.clamp(0, 90)` to prevent transmitting out-of-bounds mechanical requests.

4.2 WiFi Provisioning

The service allows provisioning the ESP32 for Station (STA) mode to connect to a local network.

```
{
  "event": "set_valve_wifi",
  "wifi_data": {
    "ssid": "TargetNetworkSSID",
    "password": "SecurePassword"
  }
}
```

Listing 5: Set WiFi Credentials Command

Critical Behavior: Sending this command forces the ESP32 to save the credentials to Non-Volatile Storage (NVS) and immediately trigger an `esp_restart()`. Consequently, the WebSocket connection will be immediately severed (going away), and the Dart client must handle the `onDone` socket event appropriately.

5 Cleanup and Resource Management

To prevent memory leaks and dangling socket connections, the `dispose()` method must be invoked when the service is no longer needed. This method sends a standard WebSocket `status.goingAway` closure frame to the server and cleanly closes all active stream controllers.